

66 作业 3 Unix V6++, malloc、free 和 栈自动扩展

同济大学计算机系操作系统 国豪 2023 拔尖班 2023

姓名 毛星博

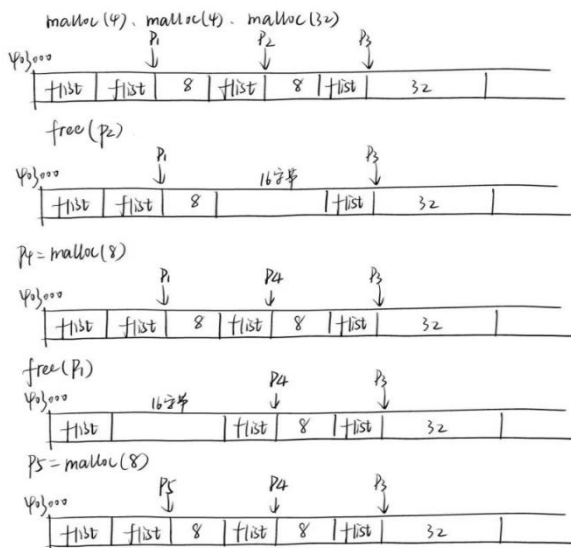
学号 2351520

一、malloc 和 free

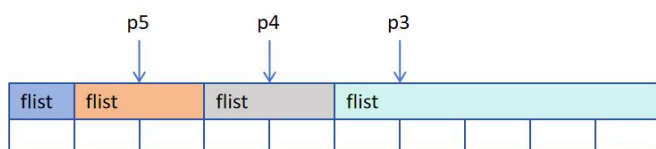
1、现运行进程 PA，1 页代码，1 页数据，没有只读数据 和 bss，1 页堆栈。代码段起始 0x401000。进程依次执行下列动态内存分配释放操作。

- char *p1= malloc(4); (1) 指针 p1 的值是多少 ? **p1 = 0x403010**
char *p2= malloc(4); (2) 指针 p2 的值是多少 ? **p2 = 0x403020**
char *p3= malloc(32); (3) 指针 p3 的值是多少 ? **p3 = 0x403030**
free(p2);
char *p4= malloc(8); (4) 指针 p4 的值是多少 ? **p4 = 0x403020**
free(p1);
char *p5= malloc(8); (5) 指针 p5 的值是多少 ? **p5 = 0x403010**
(6) 情景分析

p5 执行 malloc，发现哑元和分配给 p4 的内存片之间的空闲区（16 字节）刚好够用，用该空闲区装新内存片（没有浪费空间），返回地址是 0x403010，和原先的 p1 相同。如下图所示：



(7) 画最终的堆结构图

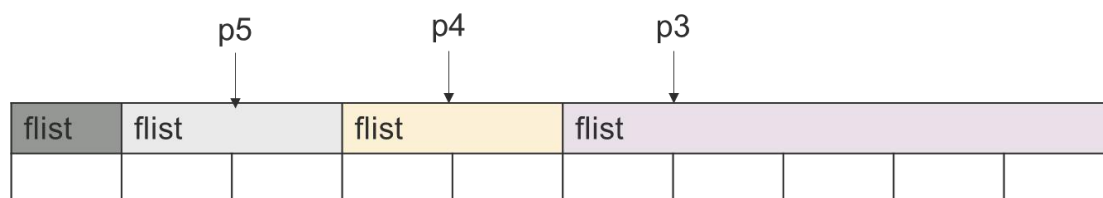


参考答案:

p1 = 0x403010 (内存片浪费 4 字节)
p2 = 0x403020 (内存片浪费 4 字节)
p3 = 0x403030
p4 = 0x403020 (p2 释放的内存片刚好够用)
p5 = 0x403010 (同上)

p5 执行 malloc，发现哑元和分配给 p4 的内存片之间的空闲区（16 字节）刚好够用，用该空闲区装新内存片（没有浪费空间），返回地址是 0x403010。

最终的堆结构图：



2、阅读 `src/program/malloc.c`。手推每块动态内存的起始地址，验证程序的输出。

Idx	Name	Size	VMA	LMA	File off	Algn
0	.text	00002808 CONTENTS,	00401000 ALLOC, LOAD,	00401000 READONLY, CODE	00000400	2**2
1	.data	00000308 CONTENTS,	00404000 ALLOC, LOAD, DATA	00404000	00002e00	2**5
2	.rdata	000003c8 CONTENTS,	00405000 ALLOC, LOAD, READONLY, DATA	00405000	00003200	2**3
3	.bss	00000030 ALLOC	00406000	00406000	00000000	2**2
4	.idata	00000014 CONTENTS,	00407000 ALLOC, LOAD, DATA	00407000	00003600	2**2
5	.stab	000052d4 CONTENTS,	00408000 READONLY, DEBUGGING, EXCLUDE	00408000	00003800	2**2
6	.stabstr	0000198a CONTENTS,	0040e000 READONLY, DEBUGGING, EXCLUDE	0040e000	00008c00	2**0

用 `objdump -h` 调试 `malloc` 程序，得到代码段 3 页，数据段 1 页，只读数据段 1 页，`bss` 段 1 页。 $0x401000+0x6000=0x407000$ 。所以堆起始地址为 `0x407000`。

```
int *c = malloc(128);
printf("the address of c: %x\n", c);
int *d = malloc(256);
printf("the address of d: %x\n", d);
int *e = malloc(256);
printf("the address of e: %x\n", e);
free(d);
int *f = malloc(83);
printf("the address of f: %x\n", f);

printf("the end of heap: %x\n", sbrk(0));

f = malloc(3*4096);
printf("the end of heap(after another expand): %x\n", sbrk(0));
```

int *c = malloc(128);哑元 8 字节, flist8 字节, 所以 $c=0x407000+0x8+0x8=0x407010$ 。
int *d = malloc(256);c 指向的数据 128 字节, flist8 字节,

$d=0x407010+0x8+0x80=0x407098$ 。

`int *e = malloc(256);` `d` 指向的数据 256 字节, `flist8` 字节,

$e=0x407098+0x8+0x100=0x4071a0$ 。

`free(d);` 从 `0x407090` 到 `0x407198` 的区域 (264 字节) 变为空闲区。

`int *f = malloc(83);` 内存片大小 $88+8=96$ 字节, 小于 264 字节, 可以填入空闲区,

$f=d=0x407098$

`printf("the end of heap: %x\n", sbrk(0))`。`sbrk(0)` 返回当前堆的尾部地址 (`malloc_end`), 由于之前的分配都没有超出第一次 `malloc` 时扩展的 3 页 (0x3000 字节) 范围, 堆没有再次扩展。 $0x407000+0x003000=0x40a000$ 输出值为 `0x40a000`。

`f = malloc(3*4096);` 所需空间为 $4096*3+8$ 。`malloc` 搜索空闲区。`d` 分割后剩下的空闲区和堆尾部的空闲区大小都不足 3 页, 所以需要调用 `sbrk()` 扩展堆, 扩展 3 页空间。所以新的堆空间尾部为 $0x40a000+0x003000=0x40d000$ 。输出值为 `0x40d000`。

通过程序输出验证所推结果无误:

```
[/bin]#malloc
The address of version is 4040f4
test of malloc
the address of c: 407010
the address of d: 407098
the address of e: 4071a0
the address of f: 407098
the end of heap: 40a000
the end of heap(after another expand): 40d000
```

- 3、阅读、逐行注释源代码 `src/lib/malloc.c`。阅读 `src/proc/Process.cpp`, `SBreak()` 函数。以应用程序 `src/program/malloc.c` 的执行过程为背景, 介绍 Unix V6++ 应用程序, 动态内存分配器的工作原理。

阅读、逐行注释源代码 `src/lib/malloc.c`:

```
#include <sys.h>
#include <malloc.h>
#include <stdio.h>

// 定义堆扩展时使用的页面块大小。
// 12288 字节 = 3 * 4096 字节, 即每次扩展 3 个标准页。
#define PAGE_SIZE 12288

// 全局指针, 用于跟踪整个堆区的起始和结束虚拟地址。
char *malloc_begin = NULL;
char *malloc_end = NULL;

// 内存片 (Chunk) 的头结构。每个由 malloc 分配的内存块前都带有一个这样的结构。
typedef struct flist {
    unsigned int size;        // 记录这个内存片的总大小 (包括头结构本身和用户可用数据区)。
    struct flist *nlink;      // 指向下一个 *已分配* 内存片的头结构的指针, 形成一个链表。
};

// 指向内存片链表头部的全局指针。
// 链表的第一个节点是一个“哑元节点” (dummy node), 用于简化链表操作。
struct flist *malloc_head = NULL;
```

```

// malloc 函数：分配 'size' 字节的动态内存。
void* malloc(unsigned int size)
{
    // 只在程序生命周期中第一次调用 malloc 时执行。
    if (malloc_begin == NULL)
    {
        // 1、首次执行 malloc，启用并初始化堆区。
        // 调用 sbrk(0) 获取当前程序数据段的末尾地址（program break），作为堆的起始点。
        malloc_begin = sbrk(0);
        // 调用 sbrk(PAGE_SIZE) 将 program break 增加 PAGE_SIZE (12288) 字节，
        // 从而扩展堆区，获得初始的堆内存。返回值是新的 program break 地址。
        malloc_end = sbrk(PAGE_SIZE);
        // 将链表头指针指向堆的起始地址。
        malloc_head = malloc_begin;
        // 初始化哑元节点。它的大小就是它自身头结构的大小（8 字节）。其作用是让链表永远不为空，简化插入
        // 和遍历逻辑。
        malloc_head->size = sizeof(struct flist);
        // 初始时，链表中只有哑元节点，所以它的 nlink 指向 NULL。
        malloc_head->nlink = NULL;
    }

    // 如果用户请求分配 0 字节，直接返回 NULL，因为没有实际意义。
    if (size == 0)
    {
        return NULL;
    }

    // 计算实际需要的内存片总大小：用户请求的大小 + 头结构的大小。
    size += sizeof(struct flist);
    // 8 字节对齐：将总大小向上取整到最近的 8 的倍数。
    size = ((size + 7) >> 3) << 3;

    // 从链表头部（哑元节点）开始遍历，寻找合适的空闲区。
    struct flist *iter = malloc_head;
    // 2、分配动态内存：在已分配的内存片之间寻找一个足够大的空闲区。
    struct flist *temp;
    while( iter->nlink != NULL )
    {
        // 遍历链表，直到最后一个节点。
        // 计算当前内存片(iter)的末尾和下一个内存片(iter->nlink)的开头之间的空闲区大小。
        if ((int)(iter->nlink) - iter->size - (int)iter >= size)
        {
            // 找到了一个足够大的空闲区，在这个空闲区的起始位置创建一个新的内存片头 temp。

```

```

        // 新内存片的位置 = 当前内存片的起始地址 + 当前内存片的大小。
        temp = (char *)iter + (iter->size);
        // 将新内存片插入链表: 新内存片的 nlink 指向原本的下一个内存片。
        temp->nlink = iter->nlink;
        // 当前内存片的 nlink 指向这个新插入的内存片。
        iter->nlink = temp;
        // 设置新内存片的大小。
        temp->size = size;
        // 返回给用户的地址是头结构之后的数据区起始地址。
        return (char *)temp + sizeof(struct flist);
    }

    // 当前空闲区不够大, 继续遍历下一个内存片。
    iter = iter->nlink;
}

// 如果循环结束仍未找到合适的空闲区, 说明所有内存片之间的空隙都不够大。
// 现在检查堆末尾 (最后一个内存片之后) 的剩余空间是否足够。
int remain;
L1: // 这是一个标签, 用于在扩展堆后跳转回来继续分配逻辑。
    // 此时 iter 指向链表的最后一个内存片。计算它到堆末尾的剩余空间大小。
    remain = malloc_end - iter->size - (int)iter;
    if ( remain >= size)
    {
        // 堆末尾的空间足够, 在最后一个内存片之后创建新的内存片 temp。
        temp = (char *)iter + (iter->size);
        // 这是新的最后一个内存片, 所以它的 nlink 为 NULL。
        temp->nlink = NULL;
        // 原本的最后一个内存片 iter 现在指向这个新的内存片。
        iter->nlink = temp;
        // 设置新内存片的大小。
        temp->size = size;
        // 返回用户可用的数据区地址。
        return (char *)temp + sizeof(struct flist);
    }

    // 3、内存分配不成功: 堆末尾空间也不足, 必须执行 sbrk 系统调用来扩展堆空间。
    // 计算还需要多少字节。
    int expand = size - remain;
    // 将需要扩展的大小向上取整到 PAGE_SIZE (12288 字节) 的整数倍。
    expand = ((expand + PAGE_SIZE - 1) / PAGE_SIZE) * PAGE_SIZE;
    // 调用 sbrk() 扩展堆, 并更新堆的末尾地址。
    malloc_end = sbrk(expand);
    // 扩展成功后, 使用 goto 跳转回 L1 处的分配逻辑, 使用刚刚获得的
    goto L1;
}

// free 函数: 释放以 addr 为首地址的动态内存。

```

```

int free(void *addr)
{
    // 用户传入的 addr 是数据区的起始地址。我们需要找到它前面的头结构。
    // 头结构大小为 8 字节，所以内存片的实际起始地址是 addr - 8。
    char *real_addr = addr - 8;
    // 从链表头部开始遍历。
    struct flist *iter = malloc_head;
    // 需要一个指针 last 来记录 iter 的前一个节点，以便执行删除操作。
    struct flist *last;
    // 如果用户传入一个空指针，这是无效操作，返回错误码。
    if (addr == 0)
    {
        return -1;
    }
    while (iter) // 遍历整个已分配内存片的链表。
    {
        // 判断当前遍历到的内存片 iter 是否是我们要释放的那个。
        if (iter == real_addr)
        {
            // 2、找到了要释放的内存片 iter。
            // 将 iter 从链表中“删除”：让前一个节点 last 直接指向 iter 的下一个节点。
            // 这样，iter 所占用的空间就变成了两个已分配内存片之间的一个“空闲区”，
            // 在下次 malloc 时可以被重新利用。
            last->nlink = iter->nlink;
            // 3、检查是否可以收缩堆：只有当被删除的是链表中的最后一个内存片时，才有可能。
            if (last->nlink == NULL)
            {
                // 计算堆末尾空闲区的起始地址。
                char *pos = (char *)last + last->size;
                // 如果删除后，堆末尾的空闲区尺寸大于一个阈值（这里是 2 倍 PAGE_SIZE，即 6 页），
                // 就调用 sbrk 收缩堆，将内存返还给操作系统。
                if (malloc_end - pos > PAGE_SIZE * 2)
                {
                    // 调用 sbrk 传入一个负值来收缩堆。收缩的大小向下取整到 PAGE_SIZE 的倍数。
                    malloc_end = sbrk(-((malloc_end - pos) / PAGE_SIZE * PAGE_SIZE));
                }
            }
            // 释放成功，返回 0。
            return 0;
        }
        // 当前节点不是要找的，继续向后遍历。
        // last 跟上 iter 的当前位置。
        last = iter;
        // iter 移动到下一个节点。
    }
}

```

```

        iter = iter->nlink;
    }

    // 如果循环走完都没找到 addr 对应的内存片，说明传入的地址是无效的（比如重复释放、野指针等）。
    // 返回错误码。
    return -1;
}

```

动态内存分配器的工作原理：

用户态 C 库 (malloc): malloc 函数是用户态内存分配的“前端”。它自身管理一块由内核批发给进程的大块内存，称之为“堆” (Heap)。当应用程序调用 malloc(size) 请求内存时，malloc 库首先检查自己管理的堆内存中是否有足够大的空闲块来满足请求。

如果有，它会从空闲块中切分出所需大小的内存，记录下这块内存已被占用，并返回其指针给应用程序。如果现有堆内存不足，malloc 库就会调用 sbrk() 系统调用，向内核申请扩大堆的边界。

内核 (sbrk 系统调用 / SBreak 函数): sbrk 是连接用户态和内核态的桥梁。当 malloc 库需要更多内存时，它会调用 sbrk(increment)，请求内核将进程的“program break”位置（堆的顶端）增加 increment 字节。

内核接收到此请求后，由 Process::SBreak() 函数处理。SBreak() 负责扩展进程的逻辑内存空间（数据段），并确保有足够的物理内存映射到这个新的地址空间。成功后，内核返回新的 program break 地址，malloc 库现在就有了更大的堆空间可供管理和分配。

4、修改 src/lib/malloc.c，将 free() 函数的时间复杂度降为 $O(1)$ 。

当前的实现之所以是 $O(n)$ ，是因为 free() 在删除一个内存片 iter 时，需要知道它的前一个内存片 last，以便执行 last->nlink = iter->nlink;。而找到 last 需要从头遍历。

使用双向链表可以避免遍历整个内存片链表来执行删除操作。

```

// 修改 flist 结构体为双向链表节点
typedef struct flist {
    unsigned int size;

    struct flist *nlink; // 指向下一个内存片
    struct flist *plink; // (新增) 指向上一个内存片
};

// 修改 free 函数，实现 O(1) 删除
int free(void *addr)
{
    if (addr == 0)
    {
        return -1;
    }

    // 定位到要释放内存片的头结构
    struct flist *block_to_free = (struct flist *)((char *)addr - sizeof(struct flist));

    // O(1) 删除操作：直接通过 plink 和 nlink 修改前后节点的指针
    struct flist *prev = block_to_free->plink;
    struct flist *next = block_to_free->nlink;

    if (prev != NULL)

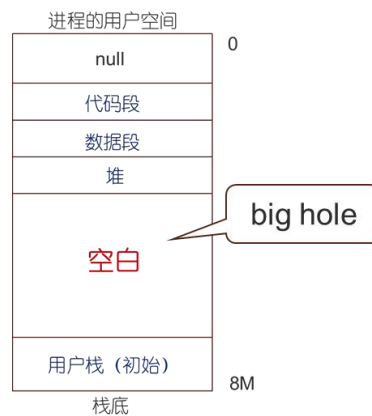
```

```

{
    prev->nlink = next; // 前一个节点的 nlink 指向下一个节点
}
if (next != NULL)
{
    next->plink = prev; // 下一个节点的 plink 指向前一个节点
}
// 检查是否可以收缩堆（只有在释放最后一个内存片时才有可能）
if (block_to_free->nlink == NULL)
{
    // 此时 prev 就是最后一个内存片
    char *pos = (char *)prev + prev->size;
    if (malloc_end - pos > PAGE_SIZE * 2)
    {
        malloc_end = sbrk(-((malloc_end - pos) / PAGE_SIZE * PAGE_SIZE));
    }
}
return 0;
}

```

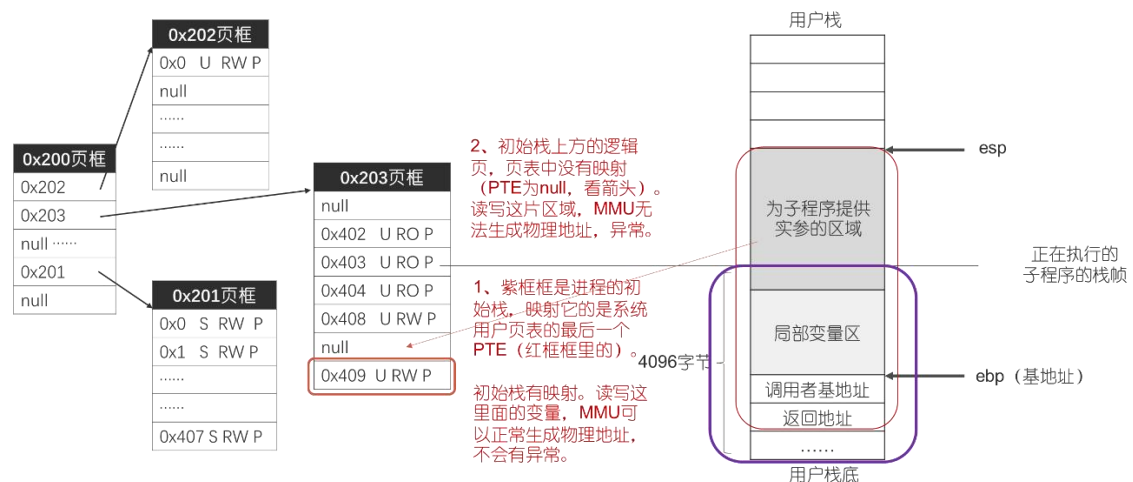
[跋] 每个需要使用动态内存的进程，有一个运行在用户空间的动态内存分配器。malloc() 和 free()，维护 flist 列表，管理从 big hole 批发来的用户堆空间。



推荐，阅读《深入理解计算机系统》 section 9.9 动态内存分配。这部分独立成篇，可以不看前后章节。

二、栈扩展

1、栈扩展，判断条件



如图，试问，现在正在执行的子程序

- (1) 访问局部变量，MMU 会抛出缺页异常嘛？

不会。因为这里有映射。如图所示，正在执行的子程序的栈帧（包括局部变量区、调用者基地址、返回地址）完全位于初始栈（紫色框）内。这块区域是有页表映射的（PTE 0x409 U RW P），因此 MMU 可以成功将虚拟地址转换为物理地址，不会抛出缺页异常。

- (2) 为子程序提供实参，~~~~？

会。因为，实参写在 esp 下面一点点。这里没有映射。为下一个子程序提供实参是通过 push 指令将参数压入栈中。push 操作会使栈顶指针 esp 向低地址移动。当 push 第一个参数时，esp 就会移动到未映射的区域（红色框）。当 CPU 尝试向这个新的 esp 地址写入数据时，MMU 发现对应的 PTE 为 null，就会立即抛出缺页异常。

- (3) 执行 call 指令，~~~~？

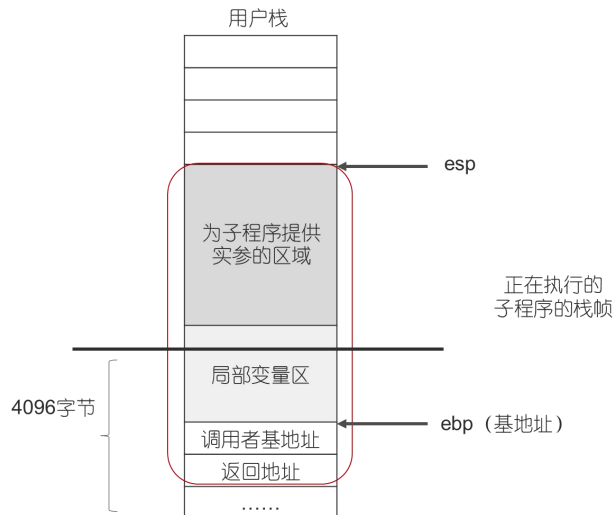
不会。缺页异常在第 (2) 步压入第一个实参 (push) 时就已经发生了。操作系统会进行栈扩展。当执行到 call 指令时，它需要将“返回地址”压入栈中。此时的栈顶 (esp) 指向的区域（红色区域内）已经被操作系统映射好了，因此 call 指令不会再触发缺页异常。

- (4) 调用的子程序执行指令 push %ebp，MMU 会抛出缺页异常吗？

不会。push %ebp 会将调用者（父程序）的基地址 ebp 压入栈中，这个位置仍然位于刚刚在第 (2) 步中由操作系统分配并映射的那个新页框（红色区域）内。既然该区域已经被映射，MMU 就可以正常翻译地址，不会抛出缺页异常。

子程序 访问局部变量，MMU 会抛出缺页异常吗？

不会。当前执行的子程序的局部变量区位于初始栈（紫色框）内，这部分区域已经有映射（PTE 0x409），MMU 可以正常工作。



如图，试问，图示栈帧

(1) 构造阶段，MMU 会抛出缺页异常嘛？

不会。构造栈帧时，只是把 esp 的值减小，还没有写入数据，不会引发缺页异常。

(2) 对应的子程序访问局部变量，MMU 会抛出缺页异常嘛？

如果访问黑线以下的（高地址区）的局部变量，不会引发缺页异常；如果访问黑线以上的（低地址区）的局部变量，会引发缺页异常。

2、完成以下任务

(1) 阅读 src/program/stack.c，观察堆栈扩展过程。

2、选这个，调试Unix V6++内核

```

253 User& u = Kernel::Instance().GetUser();
254 Process* current = u.u_proc;
255 MemoryDescriptor& md = u.u_MemoryDescriptor;
256
257 unsigned int cr2;
258 __asm__ __volatile__ ("mov %%cr2, %0" : "=r" (cr2) );
259
260 /*由缺页异常处理程序每次扩展一页，如果合理的缺了多堆栈页面，那就多执行几次缺页异常，直到把这些页面补齐*/
261
262 if ( (context->ucs & USER_MODE) == USER_MODE )
263 {
264     if ( cr2 < MemoryDescriptor::USER_SPACE_SIZE - md.m_StackSize && cr2 >= context->esp - 8
265         && md.m_DataSize + md.m_StackSize + PageManager::PAGE_SIZE < MemoryDescriptor::USER_SPACE_SIZE - md.m_DataStartAddress )
266     {
267         current->SSStack();
268     }
269     else
270     {
271         Diagnose::Write("Invalid MM access");
272         current->PSignal(User::SIGSEGV);
273         if ( current->IsSig() )
274             current->Psig( (pt_context *) &context->esp );
275     }
276 }
277 else
278     Utility::Panic("Page Fault in Kernel Mode.");
279
280

```

1、下断点在这里，只要有异常，就会在这里停下来

注释1：这是cr2寄存器，它的值是引发异常的虚地址

注释2：这是异常发生时，ESP寄存器的值

```

void test(int n)
{
    int a[100];
    int i;
    if (n > 0) {
        for (i = 0; i < 100; i++)
            a[i] = i+1;
        printf("%d\n",n);
        test(n-1);
    }
    else {
        printf("0\n");
    }
}

int main1(int argc, char* argv[])
{
    printf("%d\n",sizeof(int));

    test(100);

    printf("over\n");
    return 1;
}

```

如图，stack 程序会创建一个递归调用 100 次的函数 test，每一个都有 101 个局部变量，很明显会导致栈扩展。

在缺页异常的函数打上断点进入调试，运行 stack 函数：

```

//缺页异常 (INT 14) *有出错码*
IMPLEMENT_EXCEPTION_ENTRANCE_ERRCODE(PageFaultEntrance, PageFault)
//IMPLEMENT_EXCEPTION_HANDLER_ERRCODE(PageFault, "Page Fault!", User::SIGSEGV)

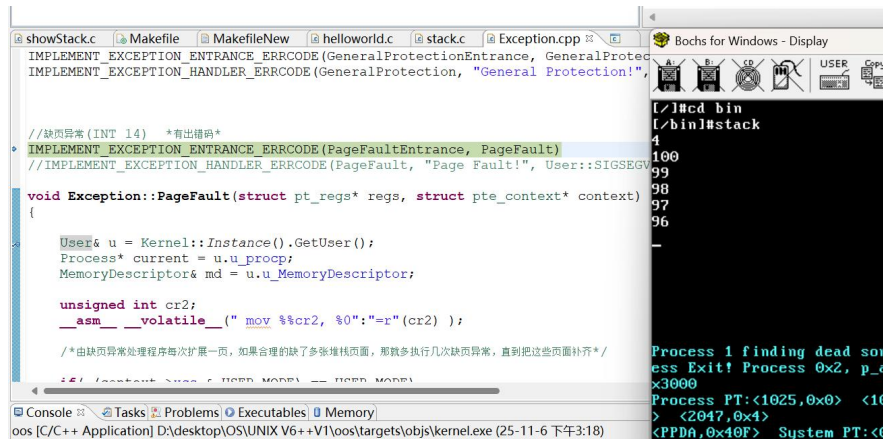
void Exception::PageFault(struct pt_regs* regs, struct pte_context* context)
{
    User& u = Kernel::Instance().GetUser();
    Process* current = u.u_proc;
    MemoryDescriptor& md = u.u_MemoryDescriptor;

    unsigned int cr2;
    __asm__ __volatile__ (" mov %%cr2, %0":"=r"(cr2) );

    /* 由缺页异常处理程序每次扩展一页，如果合理的缺了多张堆栈页面，那就多执行几次缺页异常，直到把这些页面补齐 */
    if (context && context->USER_MODE == USER_MODE)

```

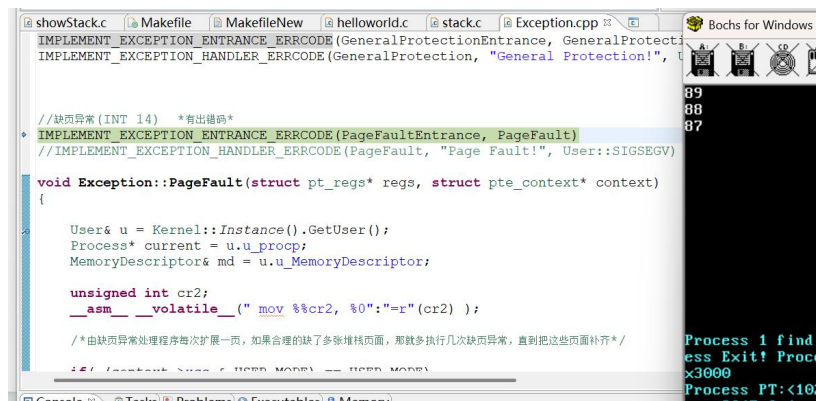
如下图所示，程序在递归了几次后就抛出了缺页异常暂停下来：



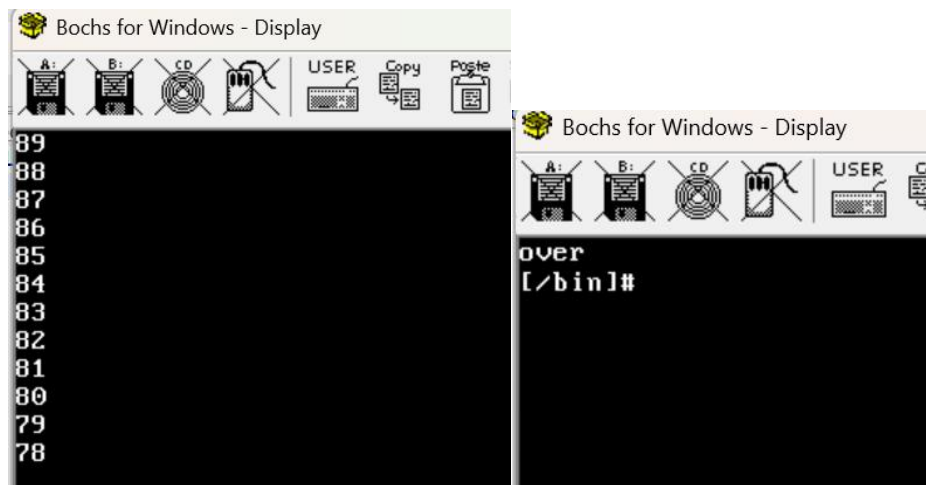
执行完缺页异常处理后，栈顶指针 esp 减小：

Name	Value
eax	4
ecx	8372608
edx	8372684
ebx	4226468
esp	0x007fbfe0
ebp	0x007fbfe0
esi	917504
edi	65452
eip	0x0040171a
eflags	[IF RF]
cs	27

程序继续往下执行，递归调用了几次 test 函数后，再次抛出缺页异常：



这样的过程还会重复多次，stack 程序才会完成执行。



(2) 堆栈扩展，判断条件是否合理？给出理由。

```
if( cr2 < MemoryDescriptor::USER_SPACE_SIZE - md.m_StackSize && cr2 >= context->esp - 8
    && md.m_DataSize + md.m_StackSize + PageManager::PAGE_SIZE <
        MemoryDescriptor::USER_SPACE_SIZE - md.m_DataStartAddress )
    current->SSStack(); // 3.1 堆栈扩展
```

合理。它通过三个与连接的子条件，确保了一次缺页异常是合法、必要且安全的栈扩展请求。

1. $cr2 < \text{MemoryDescriptor::USER_SPACE_SIZE} - md.m_StackSize$

USER_SPACE_SIZE 是用户虚拟空间的上限地址 8MB。

$md.m_StackSize$ 是当前栈已分配的大小。

$\text{USER_SPACE_SIZE} - md.m_StackSize$ 计算出的是当前栈的理论顶部（即最低的虚拟地址边界）。

这个条件确保了引发缺页的地址 $cr2$ 确实在当前栈边界的外面（更低的地址），即发生在需要扩展的区域。

2. $cr2 \geq \text{context}->esp - 8$

$\text{context}->esp$ 是异常发生时，栈顶指针 esp 的值。 $cr2$ 是引发缺页的虚拟地址。它判断导致缺页的内存访问是不是紧贴着当前栈的边界发生的。

3. $md.m_DataSize + md.m_StackSize + \text{PageManager::PAGE_SIZE} < \text{MemoryDescriptor::USER_SPACE_SIZE} - md.m_DataStartAddress$

左半部分 $md.m_DataSize + md.m_StackSize + \text{PageManager::PAGE_SIZE}$ 计算的是扩展之后，数据段（堆）和栈将占用的总虚拟空间大小。

右半部分 $\text{USER_SPACE_SIZE} - md.m_DataStartAddress$ 计算的是从数据段（堆）开始到用

户空间上限的全部可用虚拟空间。

这个条件保证了未来的总空间需求小于总可用空间。

当三个问题的答案都为“是”时，内核才会认为这是一次合法的栈扩展请求，并调用 `current->SStack()` 来执行实际的扩展操作。因此，这个判断条件是合理、安全且高效的。

(3) `cr2 >= context->esp - 4`，可不可以？

可以。这个条件意味着，内核只把发生在 `[context->esp - 4, context->esp)` 这个 4 字节区间内的缺页异常当作合法的栈增长。考虑到 Unix V6++ 所基于的 Intel i386 架构没有单条指令可以一次性 push 8 字节（64 位）的数据。常见的栈操作就是 push 一个 4 字节的值（如整数、指针、返回地址）。push 指令会先将 `esp` 减 4，然后将值存入新的 `esp` 指向的地址。因此，由 push 引发的缺页，其出错地址 `cr2` 通常就是 `esp_before_push - 4`。-4 的边界检查足以覆盖这种情况。

(4) `cr2 >= context->esp`，可不可以？

不可以。这个条件意味着，只有当缺页地址 `cr2` 大于或等于 `context->esp` 时，才被认为是合法的栈增长。但是栈的增长方向是向下的（低地址），任何合法的、需要扩展栈的操作（如 `push, call, sub esp, X`），其访问的地址 `cr2` 必然会小于 `context->esp`。如果这样改，这会让栈的自动扩展机制完全失效。

[跋] 栈是编译器和内核管的，编程的我们不需要操心。内存是宝贵的资源，先给每个进程少分点儿，真的不够用了，系统就悄悄地，后台帮你扩展。整个过程，应用程序无感，所以栈扩展和堆不一样，是自动的。